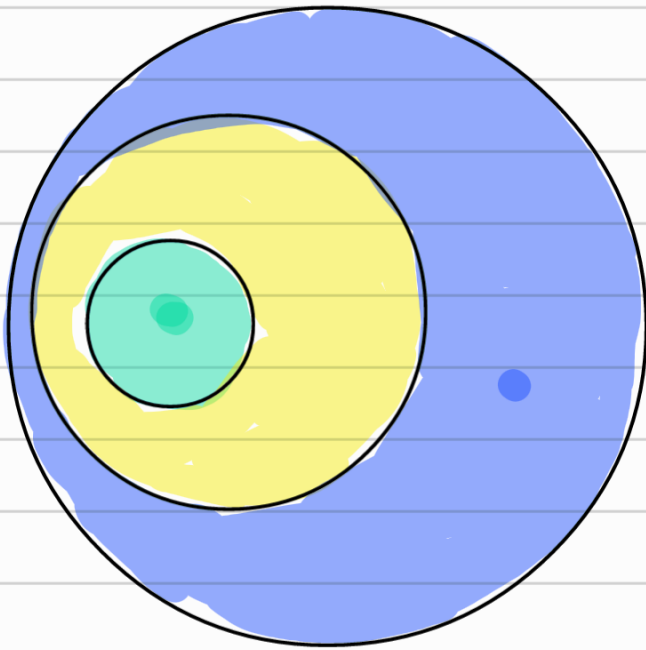


Strojové učení

- skupina algoritmů a technik, kde se na základě dostupných dat sestaví model daného systému a tento model se používá pro další předpověď budoucího chování systému
- závislost na datech



DEEP LEARNING

MACHINE LEARNING

ARTIFICIAL INTELLIGENCE

- ML souvisí s optimalizací, statistikou, data miningem

nejčastější úlohy:

- klasifikace - rozdělení dat do více tříd

(např. rozhodnout o obezitě na základě tělesných parametrů, # a druhů tříd je předem známý - obeztní, normální, ...)

- regrese - výpočet číselné hodnoty na základě vstupních parametrů, např. výpočet ceny nemovitosti na základě polohy, plochy bytu...

- shlukování (clustering)

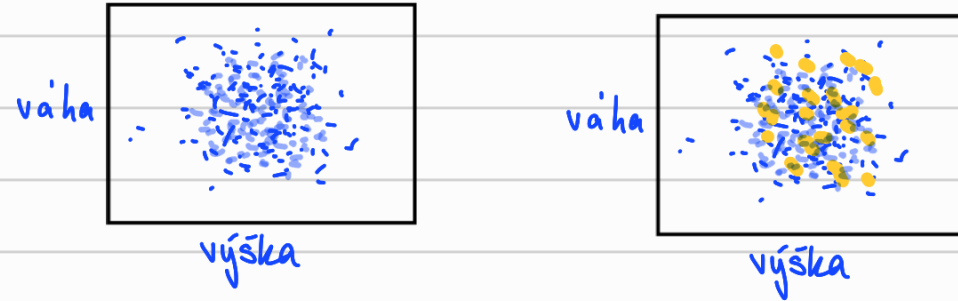
- rozdělení datových bodů do shluků (ale o třídách nevíme, zda existují)

- cílem je ověřit, zda jsou v datech podobnosti

- detekce anomálií (např. plateb kartou)
- redukce dimensionality - vybrat pouze relevantní data

např. váha, výška jedinců dána Gaussovým rozdělením

- každý jedinec je bod v příznakovém prostoru (feature space)
- scatterplot



- regrese - predikce BMI na základě váhy a výšky
- trénovací x testovací data

$$X^T \cdot X = X^T \cdot y$$

$\hat{y}$ - aproximace vektoru y

- lineární regrese - metoda nejmenších čtverců
 - kvalitu predikce hodnotíme pomocí metriky Mean Squared Error (čím menší, tím lepší) $\frac{1}{n} \sum (y - \hat{y})^2$
 - optimální situace = na obou datových množinách je podobná chyba, pokud je výrazně horší, model zobecňuje a má problém s novými testovacími daty
 - musíme explicitně odhadnout matematický model

- regrese pomocí stromů (lesů) (rozhodovací x regresní)

- výsledná předpověď je složená z dílčích předpovědí
- scikit-learn (vytvoření a naučení modelu)

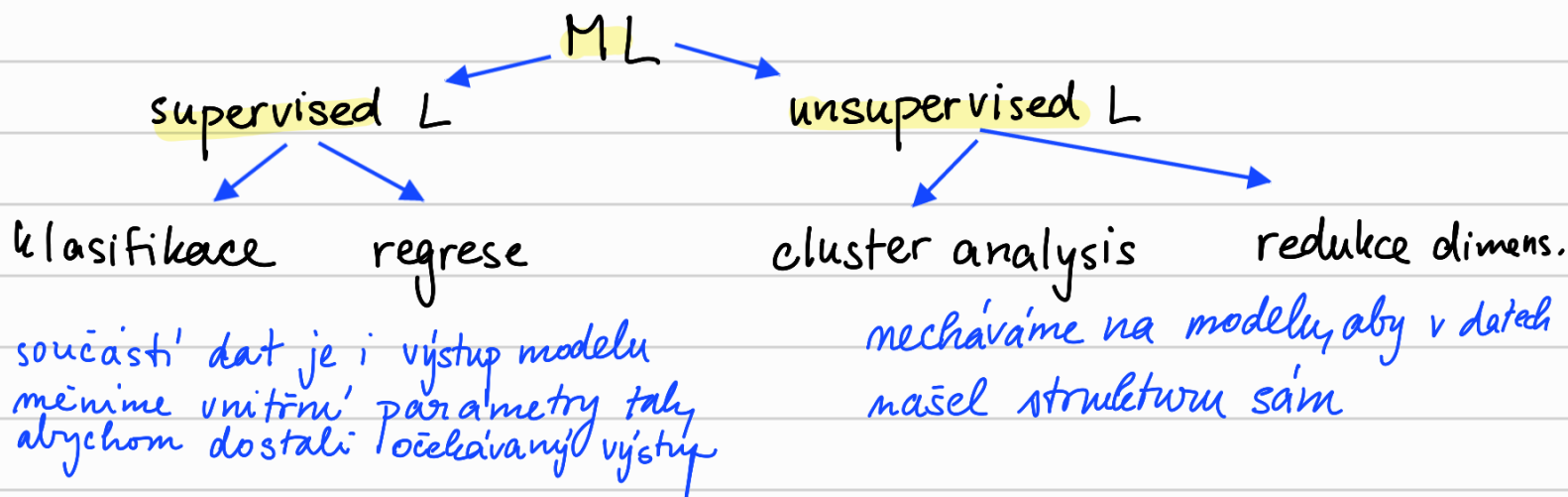
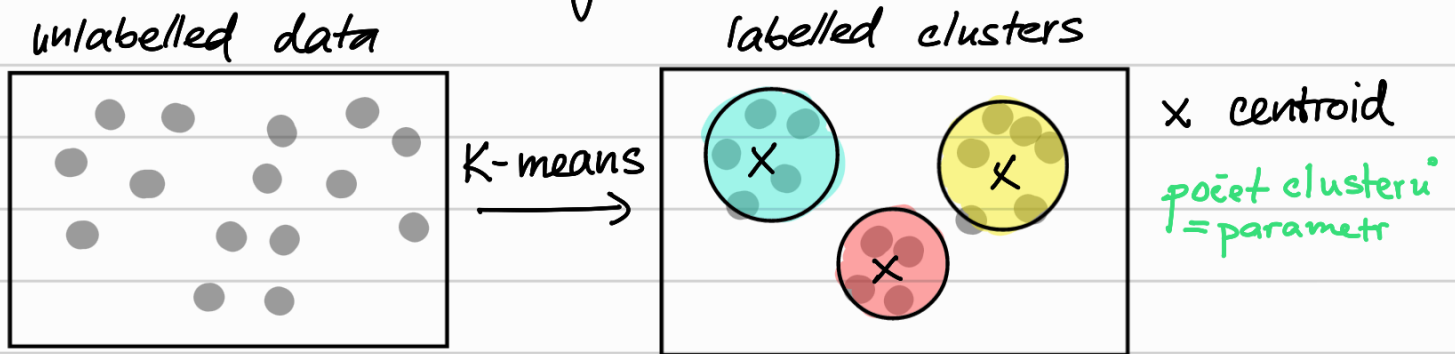
```
model = RandomForestRegressor(n_estimators=100, max_depth=8, random_state=0)
model.fit(X, y)
```

- predikce tohoto modelu je z hlediska MSE lepší
- distribuce chyby v příznakovém prostoru je lepší → model dokáže lépe předpovídat „anomálie“
- parametry v modelu = hyperparametry (nutné dobře nastavit)

• klasifikace

- binární, pokud rozdělujeme pouze do 2 tříd
- datům dodáme label, rozdělíme na trénovací a testovací
- pomocí metody **K-Nearest-Neighbor** - při klasifikaci určíme **k** nejbližších sousedů a na výsledné určíme třídu se používá ta, která převládá u nejbližších sousedů
- nutné upravit data na stejné měřítko (**standardizace**, **aby to na osách bylo stejné** **škálování**)

• shlukování (clustering)



• kčem' a data

- trénovací - nejrozsáhlejší, na nich se model učí
- validační - model závisí na hyperparametrech, tato data pro optimální nastavení
- testovací - ověřeno a ohodnoceno chováním modelu

Klasifikace

cíl: rozdělit prvky (datové body) do předem daných skupin

└ binary 0, 1

└ multiclass 0, 1, ..., n-1

└ multilabel - data mohou patřit do více tříd

- klasifikátor vrací hodnotu, občas také pravděpodobnost

ALGORITHMY

binární k. — logistická regrese

algoritmus K-NN

rozhodovací stromy (Decision Tree)

metoda podpurných vektorů (Support Vector Machine)

naive Bayes

- lineární regrese jako klasifikátor

$$y \approx X \cdot w + b$$

w - hledané váhy (metoda nejmenších čtverců)

b - bias (posouvá separační přímku)

→ řešení soustavy lin. rovnic (normální rovnice) $X^T X w = X^T y$

kritérium pro přiřazení do skupin (0, 1) $\hat{y} > 0,5$ $\hat{y} = x \cdot w$

- takto vygenerované body jsou od sebe oddělitelné přímkou (lineárně separabilní)

$$0,5 = x \cdot w + b \rightarrow x_2 = -\frac{w_1}{w_2} \cdot x_1 + \frac{b - 0,5}{w_2}$$

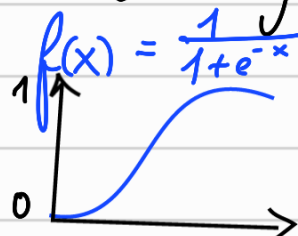
w_2 nesmí být 0, co to znamená?

- logistická regrese pro klasifikaci

• pokud chceme určit pravděpodobnost příslušnosti do třídy

$\hat{y} = \langle 0; 1 \rangle \rightarrow$ logistická fce SIGMOIDA

$$\hat{y} = \frac{1}{1 + e^{-w \cdot x}}$$



• metoda maximální věrohodnosti

(nutné používat optimalizační modely pro nalezení vah)

• metoda nejmenších čtverců (scipy)

$$\min_w \|y - \frac{1}{1 + e^{-xw}}\|^2 + c \|w\|^2 \quad \text{regularizace}$$

vektor
matice příkladů
regul.
parametr

• porozumět vzorec,
vysvětlit MNC

- 1) naučíme model pomocí metody fit()
 - 2) předpověď na dostupných datech
 - 3) vizualizace pravděpodobnosti
- při opakovaném spuštění je výsledek pokaždé odlišný

Naivní Bayesovské klasifikátory

Bayesova věta

závislé

máme 2 náhodné jevy s pravděpodobnostmi $P(A), P(B) > 0$
pro podmíněné pravděpodobnosti $P(A|B)$ a $P(B|A)$ platí

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$P(A)$ pokud B

$$P(A \cap B) = P(A) \cdot P(B|A)$$

$$\text{platí: } P(B \cap A) = P(B) \cdot P(A|B)$$

- v klasifikačním procesu máme dopředu nějakou informaci o pravděpodobnosti naší hypotézy (známe četnost jevu v popul.)
pokud obdržíme data (evidence), můžeme upravit odhad (posterior)

PŘÍKLAD: Pes, který štěká, nekouše.

H: nekousne

$$P(\text{nekousne}) = \frac{\#(\text{kouse} == N)}{\# \text{Psi}}$$

$$\text{evidence } P(\text{steka}) = \frac{\#(\text{steka} == A)}{\# \text{Psi}}$$

$$P(\text{steka} | \text{nekouse}) = \frac{\#(\text{kouse} == N \wedge \text{steka} == A)}{\#(\text{kouse} == N)}$$

~> Bayesovský vzorec

$$P(\text{nekouse} | \text{steka}) = \frac{P(\text{steka} | \text{nekouse}) \cdot P(\text{nekouse})}{P(\text{steka})} = 71\%$$

$$P(\text{nekouse} | \text{nesteka}) = \frac{P(\text{nesteka} | \text{nekouse}) \cdot P(\text{nekouse})}{P(\text{nesteka})} = 33\%$$

Ohodnocení binární klasifikace

fold body nejsou lineárně oddělit (některé patří do dvou tříd)
 - matice zaměň (confusion matrix)

realita

Klasifikátor	Pozitivní' 1	Negativní' 0
Pozitivní' 1	TP	FP
Negativní' 0	FN	TN

TP - # pozitivních, které klasifikátor správně označil

TN - # negativních, které klasifikátor správně označil

FP - negativní, ale klasifikátor označil za pozitivní

FN - pozitivní, ale klasifikátor označil za negativní

- optimální je mít prvky pouze na diagonále (sklearn)

senzitivita - $TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$

(kolik z celkového # Pozitiv. je tak správně označeno)

specificita - $TNR = \frac{TN}{N} = \frac{TN}{TN + FP}$

—||—

negativ.

$FPR = \frac{FP}{N} = 1 - TNR$

accuracy - $ACC = \frac{TP + TN}{P + N}$

Positive Predictive Value - $PPV = \frac{TP}{TP + FP}$

prevalence = $\frac{P}{P + N}$ (měření výskytu daného jevu v populaci)

Klasifikace do více tříd

ONE VS. REST (ALL) OVA

- každý klasifikátor určuje

príslušnost do jedné z n tříd

- výsledná třída je ta s největší P

ONE VS. ONE OVO

- klasifikátor pro každou dvojici tříd - větší #

- pomalejší, vhodný pro SVM

- logistická regrese (měření jistoty)

Práce s daty a datasety

- detekce chybějících hodnot (dat) - pandas isnull / isna
→ dropna
- doplnění chybějících hodnot - pandas Fillna
interpolate
replace
- scikit-learn - třída Imputer
- převod stringů na kategorické proměnné (strptime)
0, 1, 2, ...

práce s kategorickými proměnnými

- ordinální kódování Ordinal / Label Encoder
- one-hot kódování - savení nových proměnných za každou hodnotu daného příznaku (one-hot-encoder)
- převod dat na stejné měřítko (StandardScaler, MinMaxScaler)
 - fit - naučení na tréninkových datech
 - fit_transform - naučení a provedení transformace
 - transform - pouze provedení transformace (testovací data)
 - inverse_transform - inverzní transformace
- odstranění odlehklých hodnot
 - z-score - má-li daný příznak Gaussovskou distribuci, pak hodnoty s z-score > nebo < než 3 možné odstranit (pravidlo 3 sigma)
 - winsore (vliv logaritmování)

poměr rozdělení mezi trénovací a testovací data by měl být 80:20

- scikit-learn train-test-split
- vícenásobná validace, aby trénovací/testovací data nebyla pouze dobrá/špatná
→ pipelines

Dimensionalita v úlohách stroj učení

- velké množství sloupců = hodně příznaků
- čím větší dimensionalita, tím větší počet variací → potřebujeme více dat
- redukce dimensionality pomocí statistiky

kontrola míry variability pomocí rozptylu

$$\sigma^2 = \sum_{i=1}^n (x_i - E(X))^2 p_i$$

n - počet příkladů

$E(X)$ - střední hodnota

p_i - P vzorku x_i ($\frac{1}{n}$)

- ve scikit-learn `varianceThreshold`

metoda analýzy hlavních komponent (PCA)

- nebere v potaz hodnotu labelů
- singulární rozklad datasetu (SVD - v numpy)
- cílem PCA je najít takové lineární zobrazení $X \rightarrow X_{tr}$, že počet sloupců matice X_{tr} bude menší než počet sloupců matice X *dataset X je vlastně matice čísel*
- aplikováním SVD dojde ke kompresi
- princip PCA (matice kovariance)

v grafu je kovariance maximální

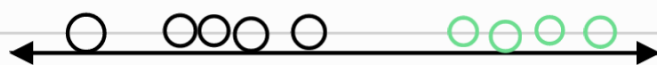
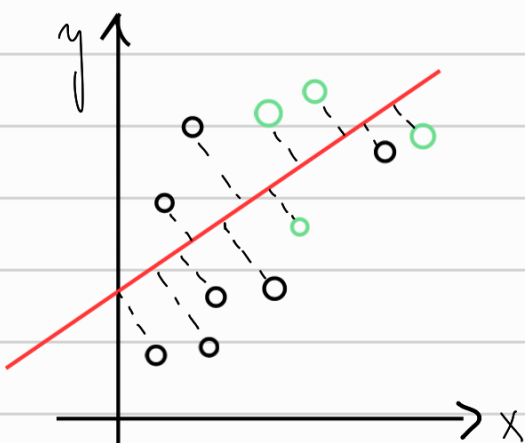


PCA po transformaci - zrcadlově otočení (1 bod stále můžeme být

- otočení (v hlavní a vedlejší osy)
- ve scikit-learn : nedochází ke standardizaci datasetu, pouze k jeho centrování (- průměr)

lineární diskriminační analýza (LDA)

- využívá labelů ($\rightarrow$ supervised ML)
- najít souřadnicovou osu takovou, že když na ni provedeme projekci datových bodů $\rightarrow$ vzdálenost mezi těžišti bude maximální a variance v rámci každé třídy minimální



Rozhodovací stromy

- supervised, vhodné pro klasifikaci i regresi
- vytvoření jednoduchých pravidel, která rozdělují příslušný prostor na menší a menší části, dokud cílová proměnná není téměř konstantní
- stromy jsou **bezparametrické** (vyjma pořadované hloubky)
- málo nároky na úpravy dat
- umí pracovat s kategoričnými i numerickými daty
- snadno interpretovatelné
- špatná schopnost generalizace při konstrukci stromů velkých hloubek - tréninková data ok, testovací nic moc (nedošlo ke správnému naučení, chová se jen jako paměť)
- konstrukce stromů - Iterative Dichotomizer 3
 - Classification & Regression Tree (CART)

principy:

- **impurita** - chceme, aby množiny byly co nejvíce stejnorodé
 - entropie (pravděpodobnost)
 - gini impurity - max pokud rovnoměrně,
0 pokud stejnorodě

výběr kritéria pro dělení

- každý rodičovský uzel má nejvýš 2 potomky
- testujeme hodnotu χ^2 a jestli má informační zisk

prořezávání stromů (pruning)

- nevýhoda: mají tendenci se přeučovat
- pokud nebudeme omezoval hloubku, bude se algoritmus chovat jako paměť na tréninková data
- není dobré, pokud výsledné listy stromů obsahují pouze jednoprvkové množiny

pre-pruning - stanoví se max. hloubka

post-pruning - napřed se vytvoří strom a pak se upravuje

seskupování stromů, lesy

algoritmus BAGGING

— Bootstrap Aggregating

— celý dataset je rozdělen na menší datasety, na každém z nich je naučen 1 dílčí klasifikátor (mohou se učit paralelně, nejsou provázané)

např. náhodné lesy (Random Forest)

algoritmus BOOSTING

— k dělení na datasety dochází postupně

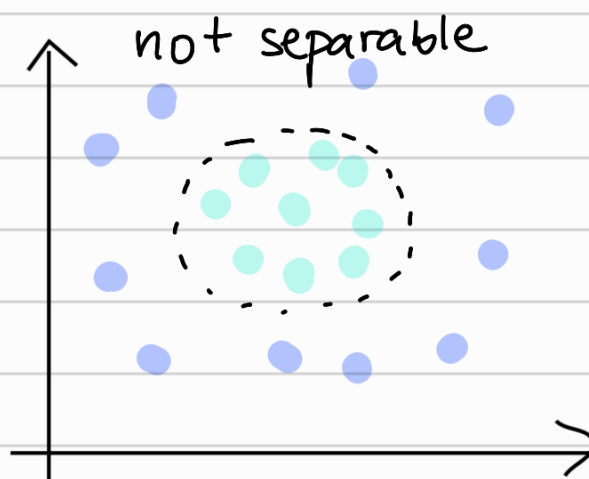
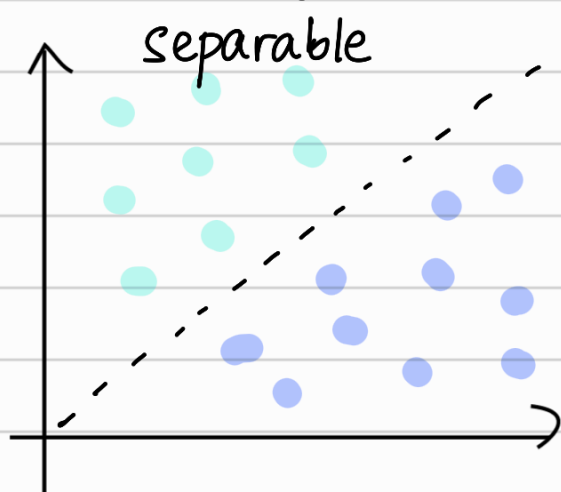
— každý další klasifikátor se učí na datech, kde předchozí selhal

— výsledky se kombinují vážením součtem dílčích

- o výsledku se rozhoduje váženým součtem všech klasifikátorů (váha je dána úspěšností klasifikátorů)
- např. ADABOOST, Gradient Tree Boosting

Lineární klasifikátory a jejich rozšíření

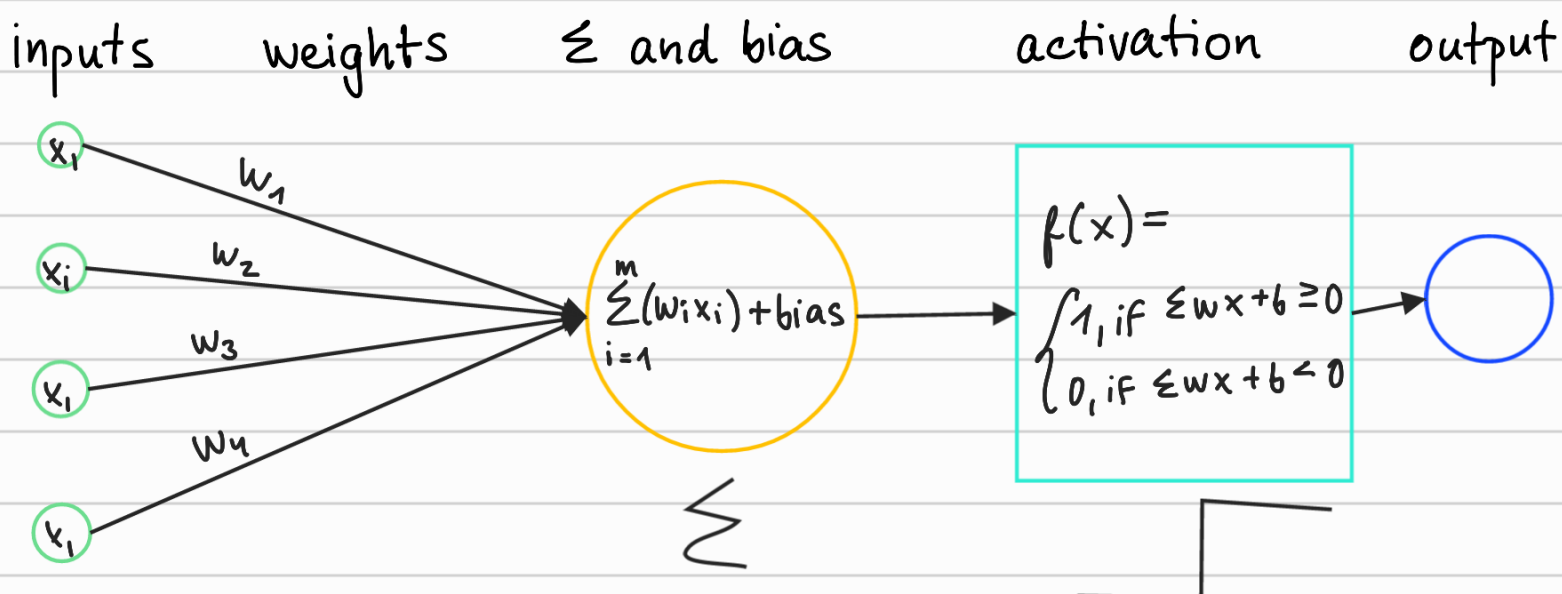
- při konstrukci se snažíme oddělit (diskriminovat) body jednotlivých tříd v příznakovém prostoru pomocí přímky ve více dimenzích



- pro zjednodušení se provádí 0-tá dimenze

Perceptron

- nejjednodušší model neuronové sítě



stratová fce - obvykle se úloha nastavení vah formuluje

jako optimalizační problém, kde se hledá min^l loss
→ na to se používá např. gradient method
(je potřeba, aby krok byl zvolen malý)

- sumace ve ztrátové funkci se dělá pouze přes jeho část (batch, minibatch)

↪ „epochy učení“

logistická regrese - v reálném světě jsou přechody mezi třídami spojitě -

minimalizace ztrátové funkce - optimalizace

- metoda nejmenších $\square$

- metoda maximální věrohodnosti (cross entropy)

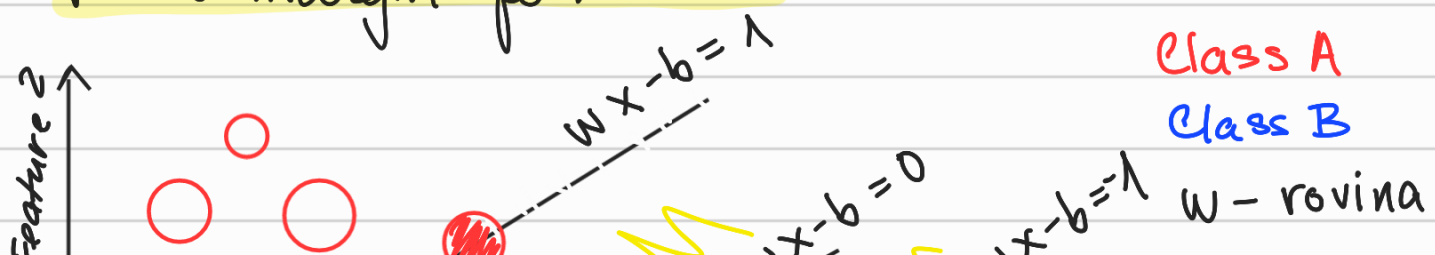
- pokud máme hodně příznaků, je logistická regrese náchylná k přecitlivění → přidat penalizační člen

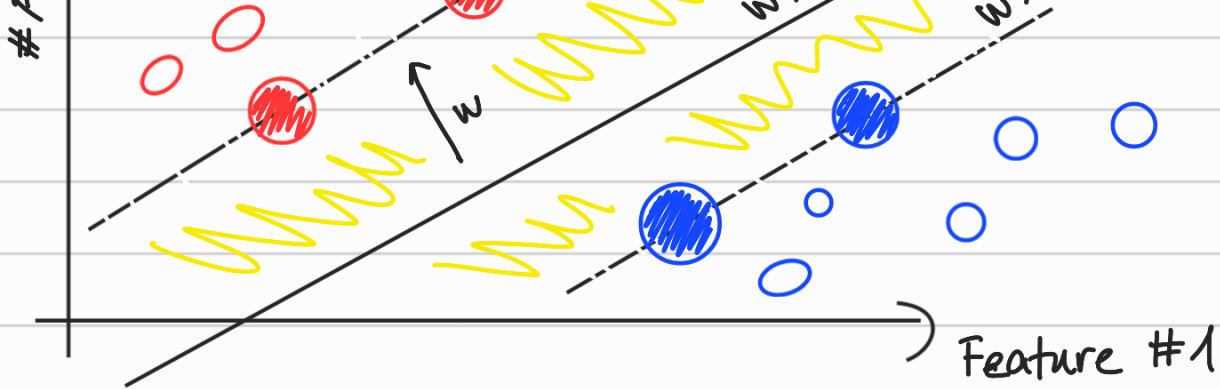
transformace pro neseparabilní případy - sklearn

Metoda podpůrných vektorů (SVM)

- rozšiřuje myšlenku lin. klasifikátorů o pojem hranice
- použitelná i pro lineárně neseparabilní úlohy
- méně výpočetně náročná než neuronové sítě

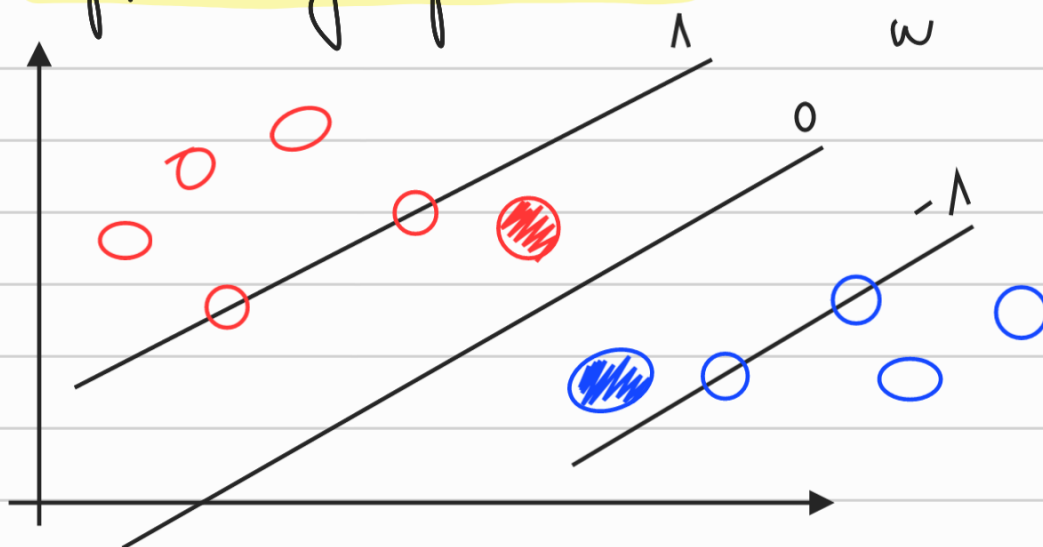
Hard-margin formulace





- data jsou lin. separable, snažíme se body separovat w
- snažíme se co nejvíc rozšířit pás v příznakovém prostoru, kde neleží žádné body
- body, které leží na hranicích, jsou podpůrné vektory

Soft-margin formulace



- dovolíme bodům, aby se nacházely pod hranicí (to se projeví v omezujících podmínkách)

Jádrová transformace - kernelizace

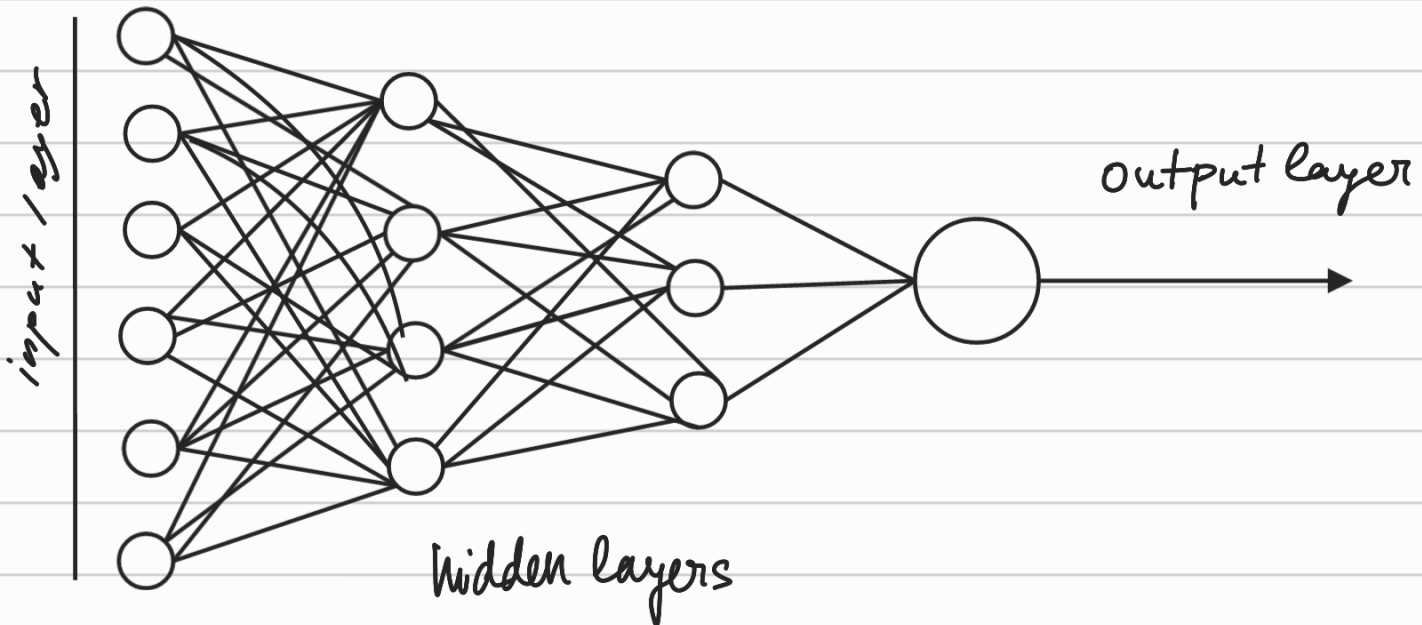
- transformace do prostoru vyšší dimenze
- udělá se jádrový trik
- jádra - lineární, polynomiální, RBF, tanh
- v SKLEARN je to výpočetně náročné

Neuronové sítě

- moderní metoda stroj. učení

Vícevrstvý perceptron MLP

- je v něm více hidden layers



- použití velkého počtu vrstev vede k velkému množství váh
- častá chyba (síť se pak chová jako paměť)
- minimalizace ztrátové funkce
- pravidlo o derivaci složené funkce (řetězové)

Aktivační funkce

- riziko, že bude vracet nulovou derivaci

Ztrátové funkce

- cross-entropy
- dataset se bere po etapách a pak se to sčítá

CNN konvoluční vrstva neuronových sítí

- pokud rozvineme obrázky do vektorů, přijdeme o topologickou informaci (pixely jsou u sebe)

Poznamky k výstupům

tréninkové obrázky: 11 090

validační: 2763 (22 tříd)

10 epoch (iterace přes celý dataset)

loss - jak dobře model predikuje výsledky

- nižší = lepší (v tréninkové fázi ok)

accuracy - kolik % vzorků bylo správně klasifikováno

- 100% už v 6. epose

val_loss, val_accuracy - na validačních datech, důležité
stoupá zlepšuje

- omezená schopnost modelu generalizovat na nová data

→ přecitlivění = overfitting

→ model opakuje chyby a ty jsou vážnější (→ loss)

ale celkový počet chyb se zmenšuje (→ přesnost)

- přidat regularizaci (aby se nepřizpůsobil šumu)

zvýšit množství dat (nebo augmentace)

Dropout (experimentovat s vyšší hodnotou)

aby se model méně přizpůsobil trénink. datům
náhodně vypne některé neurony

matrice nefunguje

Otázky ze sheetů:

- Bobbinsplot si chová více uvedených algoritmů. kdyby

trénovací množina při klasifikaci obsahovala jen prvky z
jedné skupiny

